

Минобрнауки России
Санкт-Петербургский политехнический университет Петра
Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

Раздаточный материал к докладу
«Исследование и демонстрация тестирования
идемпотентности
и повторяемости запросов в REST API на Java (Spring
Boot)»
по дисциплине «Методы тестирования программного обеспечения»

Руководитель: старший преподаватель В. А. Пархоменко
Выполнил: студент группы 5130903/30303
Плахотников Владимир Александрович

Санкт-Петербург
2026

Содержание

1	Введение	2
1.1	Актуальность	2
1.2	Постановка проблемы	2
1.3	Задачи	2
2	Методы	3
2.1	Математическая постановка задачи идемпотентности	3
2.2	Классификация HTTP-методов по идемпотентности	4
2.3	Паттерн Idempotency-Key	4
2.4	Псевдокод алгоритма проверки идемпотентности	5
2.5	Пошаговое выполнение алгоритма на иллюстративных данных	5
2.6	Типы организации контроллеров в Spring	6
2.6.1	@RestController (аннотационный подход)	6
2.6.2	Functional Endpoints (WebMvc.fn)	7
2.6.3	Spring Data REST	7
2.7	Средства тестирования	7
3	Результаты	7
3.1	Описание разработанного программного обеспечения	7
3.2	Результаты тестирования (профиль idempotent)	8
3.3	Тест-кейсы по контроллерам	8
3.3.1	@RestController (OrderIdempotencyTest)	8
3.3.2	Functional Endpoints (PaymentIdempotencyTest)	9
3.3.3	Spring Data REST (ProductIdempotencyTest)	9
3.4	Результаты профиля non-idempotent	9
4	Обсуждение	10
4.1	Сравнение подходов к обеспечению идемпотентности	10
4.2	Анализ результатов	10
4.3	Рекомендации для production	11
5	Заключение	11
6	Инструкция по установке и запуску	12
6.1	Требования	12
6.2	Запуск приложения	12
6.3	Запуск тестов	12
6.4	Пример использования API	13
	Приложение А. Ключевые фрагменты кода	15

1 Введение

1.1 Актуальность

В современных распределённых системах REST API является основным способом взаимодействия между сервисами. При обработке HTTP-запросов неизбежно возникают ситуации, когда запрос может быть отправлен повторно: из-за сетевых таймаутов, retry-политик, дублирования на уровне балансировщиков нагрузки или ошибок клиента. Без механизмов идемпотентности повторная обработка запроса приводит к критическим последствиям: дублированию платежей, двойному списанию средств, созданию дубликатов записей в базе данных [1].

Тестирование идемпотентности особенно важно в финансовых и e-commerce системах, где повторная обработка транзакции приводит к прямым финансовым потерям. Компании, такие как Stripe, реализуют паттерн Idempotency-Key как обязательный элемент API [2].

1.2 Постановка проблемы

Проблема заключается в необходимости обеспечения корректного поведения REST API при повторных запросах. HTTP-спецификация [3] определяет, какие методы должны быть идемпотентными (GET, PUT, DELETE), а какие — нет (POST). Однако на практике реализация идемпотентности требует дополнительных механизмов на уровне приложения.

Цель данной работы — исследовать и продемонстрировать методы тестирования идемпотентности REST API на примере Spring Boot приложения с тремя кардинально различными типами организации контроллеров.

1.3 Задачи

1. Формализовать понятие идемпотентности HTTP-запросов математически.
2. Разработать REST API с тремя различными типами контроллеров Spring:
 - аннотационный `@RestController` (по образцу Spring Guides [4]),
 - функциональные эндпоинты `RouterFunction` (по образцу Spring Framework [5]),
 - автогенерируемые эндпоинты Spring Data REST [6].
3. Реализовать механизмы обеспечения идемпотентности (Idempotency-Key, optimistic locking, unique constraints).

4. Разработать набор интеграционных тестов с использованием Testcontainers [7]
5. Продемонстрировать поведение системы без механизмов идиempотентности через отдельный Spring-профиль.

2 Методы

2.1 Математическая постановка задачи идиempотентности

Определение. Функция $f : X \rightarrow X$ называется *идиempотентной*, если для любого $x \in X$:

$$f(f(x)) = f(x) \quad (1)$$

В контексте HTTP API определим:

- S — множество состояний системы (совокупность данных в БД);
- R — множество HTTP-запросов;
- $O : S \times R \rightarrow S \times \text{Response}$ — операция обработки запроса;
- $\pi_1(O(s, r))$ — проекция на новое состояние.

Определение (идиempотентность HTTP-метода). HTTP-метод m называется идиempотентным, если для любого состояния $s \in S$ и запроса r с методом m :

$$\pi_1(O^n(s, r)) = \pi_1(O(s, r)), \quad \forall n \geq 1 \quad (2)$$

где O^n обозначает n -кратное применение операции, при этом каждое следующее применение использует состояние, полученное на предыдущем шаге.

Свойства. Согласно RFC 9110 [3]:

- **Идиempотентные методы:** GET, HEAD, PUT, DELETE, OPTIONS
- **Неидиempотентные методы:** POST, PATCH
- **Safe (безопасные) методы:** GET, HEAD, OPTIONS — не изменяют состояние

Формально, для safe-метода m :

$$\pi_1(O(s, r)) = s, \quad \forall s \in S, r \text{ с методом } m \quad (3)$$

Задача. Для метода POST, который не является идемпотентным по определению, необходимо обеспечить идемпотентность через дополнительные механизмы. Обозначим k — ключ идемпотентности. Тогда модифицированная операция O' :

$$O'(s, r, k) = \begin{cases} O(s, r), & \text{если } k \notin \text{KeyStore}(s) \\ (s, \text{CachedResponse}(k)), & \text{если } k \in \text{KeyStore}(s) \end{cases} \quad (4)$$

что гарантирует: $\pi_1(O'^n(s, r, k)) = \pi_1(O'(s, r, k)), \quad \forall n \geq 1$.

2.2 Классификация HTTP-методов по идемпотентности

Таблица 1: Свойства HTTP-методов (RFC 9110)

Метод	Safe	Идемпотентный	Тело запроса
GET	Да	Да	Нет
HEAD	Да	Да	Нет
PUT	Нет	Да	Да
DELETE	Нет	Да	Нет
POST	Нет	Нет	Да
PATCH	Нет	Нет	Да

2.3 Паттерн Idempotency-Key

Согласно IETF-стандарту [8], клиент передаёт уникальный ключ в заголовке **Idempotency-Key**. Сервер сохраняет этот ключ и кэширует ответ. При повторном запросе с тем же ключом сервер возвращает кэшированный ответ без повторного выполнения операции.

2.4 Псевдокод алгоритма проверки идиempотентности

Алгоритм 1 Проверка идиempотентности HTTP-запроса

Вход: HTTP-запрос *request*, хранилище ключей *KeyStore*

Выход: HTTP-ответ *response*

```
1: Функция CHECKIDEMPOTENCY(request)
2:   если request.method  $\neq$  POST тогда
3:     вернуть PROCESSREQUEST(request)      ▷ GET/PUT/DELETE
        идиempотентны
4:   конец если
5:   key  $\leftarrow$  request.headers["Idempotency-Key"]
6:   если key =  $\emptyset$  тогда
7:     вернуть 400 Bad Request      ▷ Ключ обязателен для POST
8:   конец если
9:   cached  $\leftarrow$  KeyStore.find(key)
10:  если cached  $\neq \emptyset$  и cached.response  $\neq \emptyset$  тогда
11:    вернуть cached.response      ▷ Повторный запрос —
        кэшированный ответ
12:  конец если
13:  если cached  $\neq \emptyset$  и cached.response =  $\emptyset$  тогда
14:    вернуть 409 Conflict      ▷ Запрос уже обрабатывается
15:  конец если
16:  KeyStore.register(key, request.method, request.path)
17:  response  $\leftarrow$  PROCESSREQUEST(request)
18:  KeyStore.saveResponse(key, response)
19:  вернуть response
20: конец Функция
```

2.5 Пошаговое выполнение алгоритма на иллюстративных данных

Сценарий: клиент создаёт заказ (POST /api/orders) и из-за таймаута повторяет запрос.

Входные данные:

- Запрос: POST /api/orders, тело: {"description": "Ноутбук", "amount": 89999.99}
- Заголовок: Idempotency-Key: abc-123
- Начальное состояние: таблица **orders** пуста, **KeyStore** пуст

Первый запрос (шаг за шагом):

1. Метод = POST $\Rightarrow$ проверяем Idempotency-Key
2. $key = "abc-123" \neq \emptyset \Rightarrow$ продолжаем
3. $KeyStore.find("abc-123") = \emptyset \Rightarrow$ ключ новый
4. $KeyStore.register("abc-123", POST, "/api/orders")$
5. Выполняем `ProcessRequest`: INSERT INTO orders $\Rightarrow$ заказ создан (id=e7f1...)
6. $KeyStore.saveResponse("abc-123", 201, "{id: e7f1...}")$
7. Ответ: 201 Created, тело: {id: "e7f1...", description: "Ноутбук"}

Повторный запрос (retry после таймаута):

1. Метод = POST $\Rightarrow$ проверяем Idempotency-Key
2. $key = "abc-123" \neq \emptyset \Rightarrow$ продолжаем
3. $KeyStore.find("abc-123") \neq \emptyset, cached.response \neq \emptyset$
4. Возвращаем кэшированный ответ: 201 Created, тело: {id: "e7f1..."}
5. **Заказ НЕ дублируется** — в таблице по-прежнему одна запись

2.6 Типы организации контроллеров в Spring

В данной работе используются три кардинально различных подхода к организации REST-контроллеров в Spring Framework (таблица 2).

Таблица 2: Сравнение типов контроллеров Spring

Характеристика	@RestController	Functional Endpoints	Spring REST	Data
Стиль	Аннотационный	Функциональный	Автогенерация	
Маршрутизация	@GetMapping, @PostMapping	RouterFunction	Автоматическая	
Обработчик	Метод контроллера	HandlerFunction	Репозиторий	
Бизнес-логика	В Service-слое	В Handler-классе	EventHandler	
Канонический пример	Spring Guides REST	Spring Framework docs	Spring Data REST Guide	

2.6.1 @RestController (аннотационный подход)

Классический подход, применяемый в Spring Guides [4]. Контроллер аннотируется `@RestController`, маршруты определяются через `@GetMapping`, `@PostMapping` и т. д. Обработка запроса делегируется сервисному слою.

2.6.2 Functional Endpoints (WebMvc.fn)

Функциональный подход из Spring Framework [5]. Маршруты определяются через `RouterFunction<ServerResponse>`, а обработчики — через функции `ServerRequest → ServerResponse`. Полностью отсутствуют аннотации маршрутизации.

2.6.3 Spring Data REST

Автоматическая генерация CRUD-эндпоинтов из JPA-репозитория [6]. Контроллер не пишется вручную — достаточно аннотировать репозиторий `@RepositoryRestResource`. Идемпотентность обеспечивается через `@Version` (optimistic locking) и `ETag`.

2.7 Средства тестирования

- **JUnit 5** — фреймворк модульного тестирования [9].
- **Spring Boot Test** — интеграционное тестирование Spring-приложений [10].
- **MockMvc** — тестирование HTTP-запросов без запуска сервера.
- **Testcontainers** — запуск PostgreSQL в Docker для интеграционных тестов [7].
- **Spring Profiles** — переключение конфигурации для тестирования с/без идемпотентности.

3 Результаты

3.1 Описание разработанного программного обеспечения

Разработано Spring Boot приложение, реализующее REST API для трёх доменов:

- **Заказы** (`/api/orders`) — управление заказами через аннотационный `@RestController`;
- **Платежи** (`/api/payments`) — обработка платежей через функциональные эндпоинты (`RouterFunction`);
- **Товары** (`/api/products`) — каталог товаров через автогенерируемые эндпоинты Spring Data REST.

Стек технологий: Spring Boot 3.3, Java 17, PostgreSQL 16, Flyway, Testcontainers, Docker Compose.

Структура проекта:

```
src/main/java/com/example/idempotency/  
config/          # Конфигурация (свойства, фильтры)  
idempotency/     # Механизм идиempotentности  
order/           # @RestController  
payment/         # Functional Endpoints  
product/         # Spring Data REST  
exception/       # Обработка ошибок
```

3.2 Результаты тестирования (профиль idempotent)

Основной набор из 22 интеграционных тестов проходит успешно:

Таблица 3: Результаты тестирования (профиль idempotent)

Тестовый класс	Тестов	Результат	Тип контроллера
OrderIdempotencyTest	6	PASS	@RestController
PaymentIdempotencyTest	5	PASS	Functional Endpoints
ProductIdempotencyTest	5	PASS	Spring Data REST
DataImportTest	2	PASS	Импорт JSON/CSV
NonIdempotentProfileTest	4	PASS	Демонстрация багов
Итого	22	PASS	

3.3 Тест-кейсы по контроллерам

3.3.1 @RestController (OrderIdempotencyTest)

1. **POST с одинаковым Idempotency-Key дважды** — создаётся только один заказ, второй запрос возвращает кэшированный ответ.
2. **POST с разными ключами** — создаются два разных заказа.
3. **POST без Idempotency-Key** — возвращается 400 Bad Request.
4. **PUT идиempotentен** — повторное обновление даёт тот же результат.
5. **DELETE идиempotentен** — первый раз 204, второй раз 404.
6. **GET идиempotentен** — многократные запросы возвращают одинаковый результат.

3.3.2 Functional Endpoints (PaymentIdempotencyTest)

1. **POST** платежа с одним ключом дважды — создаётся один платёж.
2. **POST** с разными ключами — создаются два платежа.
3. **POST** без ключа — 400 Bad Request.
4. **GET** идиempotentен — стабильный результат.
5. **Retry** (3 запроса) — один платёж, несмотря на 3 попытки.

3.3.3 Spring Data REST (ProductIdempotencyTest)

1. **POST** дубликата SKU — 409 Conflict (unique constraint).
2. **PUT** с корректной версией — успешное обновление, версия инкрементируется.
3. **Concurrent PUT** — optimistic locking предотвращает lost update (412 Precondition Failed).
4. **PATCH** идиempotentен — повторное применение не меняет результат.
5. **GET** идиempotentен — стабильный результат.

3.4 Результаты профиля non-idempotent

Профиль `non-idempotent` отключает механизмы идиempотентности:

- `IdempotencyFilter` пропускает все запросы без проверки;
- Unique constraint на `orders(description, amount)` удалён;
- Unique constraint на `products(sku)` удалён.

Таблица 4: Демонстрация проблем без идемпотентности (профиль non-idempotent)

Тест	Обнаруженная проблема	Статус
POST без Idempotency-Key	Запрос проходит без проверки	BUG
Повторный POST с тем же ключом	Создаётся дубликат заказа	BUG
POST с дубликатом SKU	Создаётся дубликат товара	BUG
Retry платежа (3 раза)	3 платежа вместо 1 (тройное списание)	BUG

Тесты `NonIdempotentProfileTest` проходят зелёными, подтверждая наличие проблем и демонстрируя, что без механизмов идемпотентности система работает некорректно.

4 Обсуждение

4.1 Сравнение подходов к обеспечению идемпотентности

Результаты исследования показывают, что каждый тип контроллера Spring требует различного подхода к обеспечению идемпотентности (таблица 5).

Таблица 5: Сравнение механизмов идемпотентности по типам контроллеров

Критерий	@RestController	Functional	Data REST
Механизм POST	Idempotency-Key + фильтр	Idempotency-Key + фильтр	Unique constraints
Механизм PUT	Естественная идемпотентность	Естественная идемпотентность	@Version + ETag
Concurrent защита	На уровне фильтра	На уровне фильтра	Optimistic locking
Сложность реализации	Средняя	Средняя	Низкая (автоматически)
Гибкость	Высокая	Высокая	Ограниченная

4.2 Анализ результатов

Аннотационный `@RestController` предоставляет наибольшую гибкость в реализации идемпотентности, но требует ручной реализации филь-

тра и хранилища ключей. Этот подход наиболее распространён в production-системах благодаря явности и читаемости кода.

Функциональные эндпоинты (WebMvc.fn) используют тот же механизм Idempotency-Key через общий фильтр, что демонстрирует преимущество архитектуры на основе фильтров — она работает независимо от типа контроллера. Функциональный стиль обеспечивает лучшую композиционность и тестируемость отдельных обработчиков.

Spring Data REST автоматически обеспечивает ряд механизмов идемпотентности: `@Version` генерирует ETag, а unique constraints предотвращают дубликаты. Это минимизирует объём ручного кода, но ограничивает возможности кастомизации.

4.3 Рекомендации для production

1. **Всегда требовать Idempotency-Key для POST-запросов**, создающих ресурсы или выполняющих побочные эффекты.
2. **Использовать optimistic locking (@Version)** для защиты от concurrent updates.
3. **Устанавливать TTL** для ключей идемпотентности (рекомендуется 24 часа [8]).
4. **Хранить ключи** в той же СУБД, что и бизнес-данные, для обеспечения транзакционной консистентности.
5. **Тестировать с Testcontainers [7]**, используя реальную СУБД вместо in-memory заменителей, для достоверности результатов.

5 Заключение

В данной работе проведено исследование и демонстрация тестирования идемпотентности REST API на базе Spring Boot. Разработано приложение с тремя кардинально различными типами контроллеров, для каждого из которых реализованы и протестированы механизмы обеспечения идемпотентности.

Основные результаты:

- Формализована математическая модель идемпотентности HTTP-запросов.
- Реализован и протестирован паттерн Idempotency-Key с хранением ключей в PostgreSQL.
- Разработано 22 интеграционных теста, покрывающих все аспекты идемпотентности для трёх типов контроллеров.

- Продемонстрировано критическое значение механизмов идемпотентности: при их отключении система допускает дублирование данных и тройное списание при retry.

6 Инструкция по установке и запуску

6.1 Требования

- Java 17+ (JDK)
- Maven 3.8+
- Docker (для PostgreSQL и Testcontainers)

6.2 Запуск приложения

1. Запустить PostgreSQL через Docker Compose:

```
docker-compose up -d
```

2. Собрать и запустить приложение:

```
mvn spring-boot:run
```

3. API доступно по адресу `http://localhost:8080`:

- `/api/orders` — заказы (@RestController)
- `/api/payments` — платежи (Functional Endpoints)
- `/api/products` — товары (Spring Data REST)
- `/api/explorer` — HAL Explorer (Spring Data REST)

6.3 Запуск тестов

1. Запуск всех тестов (профиль с идемпотентностью):

```
mvn test
```

2. Запуск тестов без идемпотентности (демонстрация багов):

```
mvn test -Dtest="NonIdempotentProfileTest"
```

Примечание: для запуска тестов требуется Docker, так как Testcontainers автоматически запускает PostgreSQL в контейнере.

6.4 Пример использования API

Создание заказа с Idempotency-Key:

```
curl -X POST http://localhost:8080/api/orders \  
-H "Content-Type: application/json" \  
-H "Idempotency-Key: unique-key-123" \  
-d '{"description": "Ноутбук", "amount": 89999.99}'
```

Повторный запрос с тем же ключом вернёт кэшированный ответ (заказ не дублируется).

Список литературы

1. *Helland P.* Idempotence Is Not a Medical Condition // Communications of the ACM. Т. 55. — ACM, 2012. — С. 56—65.
2. *Stripe.* Idempotent Requests — Stripe API. — 2024. — URL: https://stripe.com/docs/api/idempotent_requests ; Stripe API Documentation.
3. *Fielding R., Nottingham M., Reschke J.* HTTP Semantics. — 2022. — URL: <https://www.rfc-editor.org/rfc/rfc9110> ; Internet Engineering Task Force. RFC 9110.
4. *Spring Team.* Building REST services with Spring. — 2024. — URL: <https://spring.io/guides/tutorials/rest> ; Spring Guides.
5. *Spring Framework.* Functional Endpoints — Spring Web MVC. — 2024. — URL: <https://docs.spring.io/spring-framework/reference/web/webmvc-functional.html> ; Spring Framework Reference Documentation.
6. *Spring Team.* Accessing JPA Data with REST. — 2024. — URL: <https://spring.io/guides/gs/accessing-data-rest> ; Spring Guides.
7. *AtomicJar.* Testcontainers — Integration testing with real dependencies. — 2024. — URL: <https://testcontainers.com/> ; Официальная документация.
8. *Dalal S., Jena J.* The Idempotency-Key HTTP Header Field. — 2024. — URL: <https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/> ; draft-ietf-httpapi-idempotency-key-header-06. IETF Internet-Draft.
9. *Beck K.* Test Driven Development: By Example. — Addison-Wesley, 2002. — ISBN 978-0321146533.
10. *Spring Team.* Testing in Spring Boot. — 2024. — URL: <https://docs.spring.io/spring-boot/reference/testing/index.html> ; Spring Boot Reference Documentation.

Приложение А. Ключевые фрагменты кода

А.1. IdempotencyFilter (фильтр проверки идемпотентности)

Листинг 1: IdempotencyFilter.java — HTTP-фильтр идемпотентности

```
1 public class IdempotencyFilter extends OncePerRequestFilter {
2
3     public static final String IDEMPOTENCY_KEY_HEADER = "
4         Idempotency-Key";
5     private final IdempotencyService idempotencyService;
6     private final IdempotencyProperties properties;
7
8     @Override
9     protected void doFilterInternal(HttpServletRequest request,
10        HttpServletResponse response, FilterChain
11        filterChain)
12        throws ServletException, IOException {
13         if (!"POST".equalsIgnoreCase(request.getMethod())) {
14             filterChain.doFilter(request, response);
15             return;
16         }
17         if (!properties.isEnabled()) {
18             filterChain.doFilter(request, response);
19             return;
20         }
21         String key = request.getHeader(IDEMPOTENCY_KEY_HEADER);
22         if (key == null || key.isBlank()) {
23             response.setStatus(400);
24             response.getWriter().write("{\"error\":\""
25                 Idempotency-Key required\""}");
26             return;
27         }
28         var existing = idempotencyService.findByKey(key);
29         if (existing.isPresent() && existing.get().
30             getResponseStatus() != null) {
31             var cached = existing.get();
32             response.setStatus(cached.getResponseStatus());
33             response.getWriter().write(cached.getResponseBody()
34                 );
35             return;
36         }
37         idempotencyService.registerKey(key, request.getMethod()
38             , request.getRequestURI());
39         ContentCachingResponseWrapper wrapper = new
40             ContentCachingResponseWrapper(response);
41         filterChain.doFilter(request, wrapper);
42         String body = new String(wrapper.getContentAsByteArray
43             (), StandardCharsets.UTF_8);
```



```

36         idempotencyService.saveResponse(key, wrapper.getStatus
37             (), body);
37         wrapper.copyBodyToResponse();
38     }
39 }

```

A.2. OrderController (@RestController)

Листинг 2: OrderController.java — аннотационный контроллер

```

1 @RestController
2 @RequestMapping("/api/orders")
3 public class OrderController {
4     private final OrderService orderService;
5
6     @GetMapping
7     public List<Order> list() { return orderService.findAll();
8     }
9
10    @PostMapping
11    public ResponseEntity<Order> create(@Valid @RequestBody
12        OrderDto dto) {
13        return ResponseEntity.status(HttpStatus.CREATED).body(
14            orderService.create(dto));
15    }
16
17    @PutMapping("/{id}")
18    public Order update(@PathVariable UUID id, @Valid
19        @RequestBody OrderDto dto) {
20        return orderService.update(id, dto);
21    }
22
23    @DeleteMapping("/{id}")
24    @ResponseStatus(HttpStatus.NO_CONTENT)
25    public void delete(@PathVariable UUID id) { orderService.
26        delete(id); }
27 }

```

A.3. PaymentRouter (Functional Endpoints)

Листинг 3: PaymentRouter.java — функциональная маршрутизация

```

1 @Configuration
2 public class PaymentRouter {
3     @Bean
4     public RouterFunction<ServerResponse> paymentRoutes(
5         PaymentHandler handler) {
6         return RouterFunctions.route()
7             .path("/api/payments", builder -> builder

```

```

7         .GET("", accept(APPLICATION_JSON), handler::
            list)
8         .GET("/{id}", accept(APPLICATION_JSON), handler
            ::getById)
9         .POST("", contentType(APPLICATION_JSON),
            handler::create)
10    ).build();
11 }
12 }

```

A.4. ProductRepository (Spring Data REST)

Листинг 4: ProductRepository.java — автогенерация REST API

```

1 @RepositoryRestResource(collectionResourceRel = "products",
    path = "products")
2 public interface ProductRepository extends JpaRepository<
    Product, Long> {
3     Optional<Product> findBySku(String sku);
4 }

```

A.5. Пример теста идемпотентности

Листинг 5: Тест: повторный POST с Idempotency-Key

```

1 @Test
2 @DisplayName("POST with same Idempotency-Key twice - only one
    order created")
3 void postWithSameIdempotencyKey_shouldReturnCachedResponse()
    throws Exception {
4     var dto = new OrderDto("Laptop", new BigDecimal("89999.99"),
        null);
5     String body = objectMapper.writeValueAsString(dto);
6     String key = UUID.randomUUID().toString();
7
8     // First request - creates order
9     mockMvc.perform(post("/api/orders")
10        .contentType(MediaType.APPLICATION_JSON)
11        .header("Idempotency-Key", key)
12        .content(body))
13        .andExpect(status().isCreated());
14
15    // Second request with same key - returns cached response
16    mockMvc.perform(post("/api/orders")
17        .contentType(MediaType.APPLICATION_JSON)
18        .header("Idempotency-Key", key)
19        .content(body))
20        .andExpect(status().isCreated());
21 }

```

```
22     assertEquals(1, orderRepository.count(),
23         "Duplicate POST must not create a second order");
24 }
```